CSS3 - Grid

Diseño de Interfaces Web

¿Qué es CSS Grid?

Grid es un sistema de diseño bidimensional que redefine la manera en que estructuramos las páginas web.

Nos permite crear layouts complejos y estructurados con facilidad.

- Bidimensional: Diseña tanto en columnas como en filas.
- Precisión de Control: Permite una colocación precisa de elementos en la intersección de líneas de cuadrícula.
- Flexibilidad: Maneja desde la estructura general de la página hasta detalles finos, como espaciado y ubicación.

En resumen, CSS Grid ofrece una solución versátil y eficaz para el diseño web moderno, brindando a los desarrolladores un control sin precedentes sobre la presentación de sus contenidos.

Ventajas de CSS Grid

Ventajas:

- Mayor control sobre el diseño.
- Simplifica la creación de layouts responsivos.
- Facilita la alineación y distribución de elementos en la página.

Conceptos Básicos

Estructura Básica de Grid:

- Contenedor de Grid.

  Define el contexto para la creación del Grid.

- Elementos secundarios (Grid Items).

  Los elementos dentro del contenedor que se organizarán en el Grid.

Creación de un Grid

1. Propiedad display: grid;
2. Especificando columnas y filas

```
.container {
    display: grid;
    grid-template-columns: repeat(3, 1fr); /* Tres columnas de igual tamaño */
    grid-template-rows: 100px 200px; /* Dos filas con alturas específicas */
}
```

Propiedades del Contenedor

grid-template-columns y grid-template-rows

Especifican la cantidad y tamaño de las columnas y filas.

Ej.

```
.container {
    grid-template-columns: 1fr 2fr 1fr; /* Tres columnas, la del medio es el doble de ancha */
    grid-template-rows: repeat(3, 100px); /* Tres filas de 100px de altura */
}
```

Propiedades del Contenedor

grid-auto-columns y grid-auto-rows

Especifican el tamaño predeterminado de columnas y filas cuando no se definen explícitamente.

Ej.

```
.container {
    grid-auto-columns: 1fr; /* Se establece que todas las comunas serán iguales*/
    grid-auto-rows: 100px; /* Se establece que todas las filas que se creen serán de 100px*/
}
```

Propiedades del Contenedor

gap

Establece el espacio entre las celdas.

Ej.

```
.container {
    gap: 10px; /* Espacio de 10px entre celdas */
}
```

Colocación de Elementos en el Grid

grid-column y grid-row

- 1 valor
  - Indica el posicionamiento del elemento.
  - Entiende que solo va a ocupar 1 fila/columna

- 2 valores separados por /
  - Primer valor: posición desde la que comenzará
  - Segundo valor: posición hasta la que ocupará
    - Se indica la posición de inicio del próximo item o del espacio en blanco, es decir, +1 de la fila/columna que queremos que esté incluida.

Span

La palabra clave span permite indicar en las propiedades grid-column y grid-row cuántas columnas o filas deben abarcar los elementos de la cuadrícula, sin necesidad de indicar dónde terminan.

Sintaxis:

```
grid-column: 2 / span 2;

//comienza en la segunda línea vertical y ocupa dos columnas

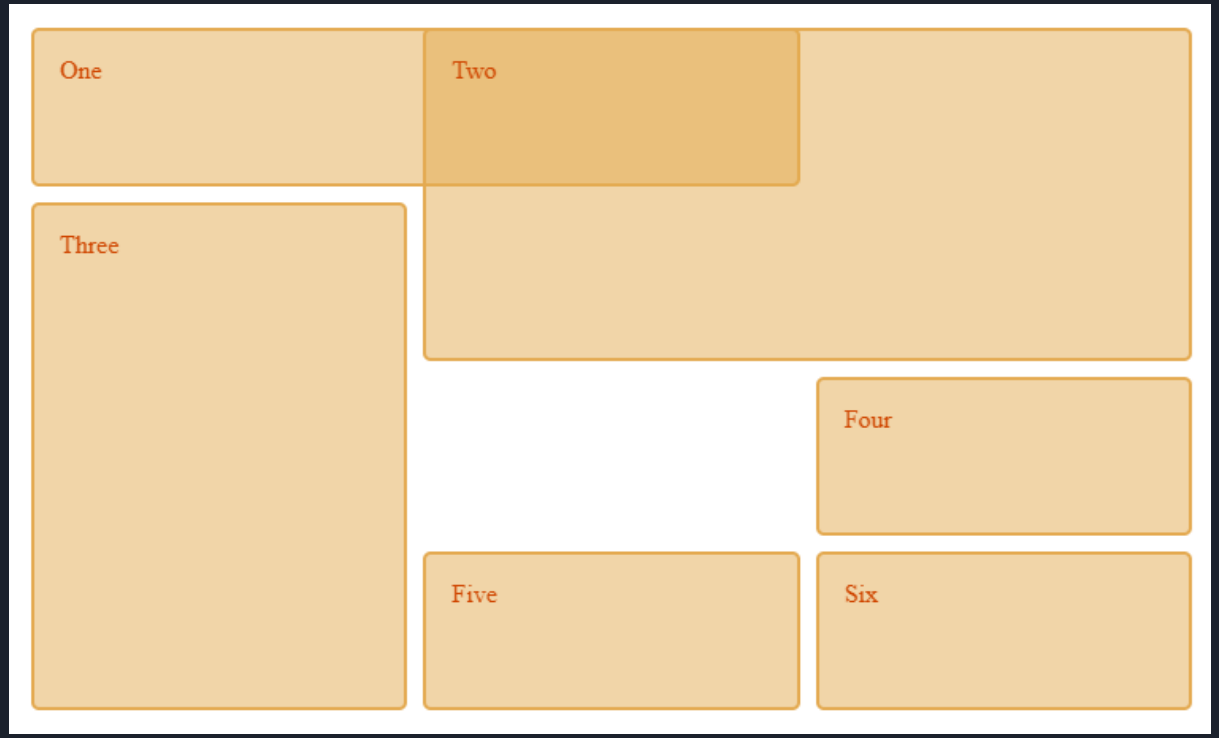

grid-row: 3 / span 1;

//comienza en la segunda línea horizontal y ocupa una fila.
```

Colocación de Elementos en el Grid

Ejemplo:

```css
.wrapper {
  display: grid;
  grid-template-columns: repeat(3, 1fr);
  grid-gap: 10px;
  grid-auto-rows: minmax(100px, auto);
}
.one {
  grid-column: 1 / 3;
  grid-row: 1;
}
.two {
  grid-column: 2 / 4;
  grid-row: 1 / 3;
}
.three {
  grid-column: 1;
  grid-row: 2 / 5;
}
.four {
  grid-column: 3;
  grid-row: 3;
}
.five {
  grid-column: 2;
  grid-row: 4;
}
.six {
  grid-column: 3;
  grid-row: 4;
}
```



Alineación en Grid

justify-content y align-content

```
.container {
    justify-content: space-around;  /* Espacio alrededor de las columnas */
    align-content: center;  /* Centra las filas en el eje vertical */
}
```

justify-items y align-items

```
.item {
    justify-items: center;  /* Centra el contenido horizontalmente dentro de su celda */
    align-items: end;  /*Alinea el contenido en la parte inferior de su celda */
}
```

Grid Areas

grid-template-areas permite definir áreas en la cuadrícula y asignar nombres a estas áreas.

Posteriormente, puedes asignar elementos específicos a estas áreas utilizando la propiedad grid-area.

Esto facilita la comprensión del diseño y la ubicación de elementos en la cuadrícula. Es particularmente útil en diseños complejos donde la estructura visual es esencial.

```
.container {
        display: grid;
        grid-template-columns: repeat(3, 1fr);
        grid-template-rows: auto;
        grid-template-areas:
                'header header header'
                'main   sidebar sidebar'
                'footer footer footer';
}
```

```
.header {
        grid-area: header;
}
.main-content {
        grid-area: main;
}
.sidebar {
        grid-area: sidebar;
}
.footer {
        grid-area: footer;
}
```

Grid Dinámico

El Grid Dinámico permite que las celdas de un contenedor de cuadrícula se adapten a diferentes tamaños de pantalla.

Facilita la creación de diseños responsivos sin necesidad de media queries complicadas.

minmax(*p1*, *p2*)

La función 'minmax' se utiliza para definir un tamaño mínimo y máximo. Se utiliza sobre todo para definir el ancho de las columnas.

Ejemplo: grid-template-columns: repeat(3, minmax(150px, 1fr));

Esto crea tres columnas de al menos 150px, expandiéndose hasta llenar el espacio disponible.

Ejemplo2: grid-template-columns: minmax(100px,250px) minmax(600px,1fr) minmax(50px, 100px);

Esto crea tres columnas de al menos 100px, 600px y 50px; expandiéndose hasta llegar a 250px la primera, cubrir el espacio disponible la segunda y 100px la tercera.

auto-fill

La función 'auto-fill' permite que el contenedor llene automáticamente el espacio disponible con las columnas definidas.

Se usa como primer parámetro de la función repeat.

Si hay espacio adicional, se agregan más columnas vacías.

Ejemplo:

```
.container {
    display: grid;
    grid-template-columns: repeat(auto-fill, 150px);
}
```

auto-fit

La función 'auto-fit' también llena el espacio disponible con columnas, pero hasta llegar al máximo de columnas marcado por el número de elementos contenidos.

A diferencia de 'auto-fill', no deja columnas vacías si hay espacio, sino que limita la propagación del contenedor.

También se usa como primer parámetro de la función repeat.

Ejemplo:

```
.container {
    display: grid;
    grid-template-columns: repeat(auto-fit, 150px);
}
```

auto-fill y auto-fit + minmax

Cuando de verdad toman poder estas funciones es cuando se utilizan de manera conjunta. Cuando se usan auto-fill y auto-fit con minmax, el valor mínimo de éste suele ser el que en cada caso se considere oportuno pero el máximo suele ser 1fr.

Ejemplo:

```
.container {
    display: grid;
    grid-template-columns: repeat(auto-fill, minmax(150px, 1fr));
}
```

En este ejemplo se irán generando tantas columnas como sea posible teniendo un ancho mínimo de 150px y máximo de lo que el tamaño de pantalla permita. Si se va aumentando el tamaño de pantalla y llega un momento en que queda una columna más de 150px, todas tomarán este valor y se añadirá una nueva columna.

La única diferencia de comportamiento entre auto-fill y auto-fit en estos casos se produce cuando hay menos elementos que columnas entrarían en la pantalla.

auto-fill y auto-fit + minmax

- auto-fill seguiría creando columnas de la misma manera que si hubiese elementos suficientes como para llenarlas pero dejándolas vacías.

- auto-fit aumentaría el tamaño de las columnas que contuviesen elementos hasta llenar todo el ancho del contenedor.

min-content y max-content

Ambas se utilizan para establecer el ancho de columnas en layouts.

min-content: Representa el espacio mínimo necesario para que el contenido se ajuste sin partir palabras en distintas filas. Es decir, ancho columna = ancho de palabra más larga.

max-content: Representa el espacio máximo necesario para que el contenido se muestre sin partir frases en distintas filas. Es decir, ancho columna = ancho de toda la frase.

Ejemplo:

```
.container {
    display: grid;
    grid-template-columns: min-content max-content;
}
```

La primera columna tendrá el ancho de la palabra más larga y la segunda el ancho de la frase completa.

grid-auto-flow

La propiedad grid-auto-flow controla cómo se colocan los elementos en un contenedor de cuadrícula.

Valores:

- row: (valor por defecto) Los elementos se colocan en filas.
- column: Los elementos se colocan en columnas.
- dense: Trata de llenar los espacios vacíos en la cuadrícula, colocándolos de manera más compacta.

Ejemplo:

```
.container {
    display: grid;
    grid-template-columns: repeat(3, 100px);
    grid-auto-flow: dense;}

#primero {
    grid-column: 1 / 3}

#segundo {
    grid-column: 2 / span 2}
```

En este ejemplo, #tercero iría seguidamente después de #segundo donde tuviera hueco (que como no indicamos nada ocupa 1x1), es decir, como tenemos tres columnas y #segundo abarca la segunda y la tercera, #teercero iría a la primera columna de la tercera fila. Sin embargo, al estar el grid-auto-flow: dense, #tercero ocupará el hueco de 1x1 que queda vació a la derecha de #primero.